

System Design & Architecture Masterclass

The Complete GitHub Pages Hosting, Content Protection & Viral LinkedIn Diagram Playbook

1. Why Architecture Visual Diagrams Go Viral (Save/Bookmark Psychology)

On technical LinkedIn, plain text posts get 500-1,000 views. **High-density architecture flowcharts, component blueprints, and visual comparison diagrams get 10,000+ to 100,000+ views.** Developers, engineering managers, and architects instinctively **Save and Screenshot** clean visual blueprints to reference during interview preparation or on-the-job system planning.

Every post in your `linkedin_post_assets/` directory has a dedicated high-resolution (300 DPI) architecture diagram blueprint designed specifically to trigger saves and reposts.

2. Zero-Friction Hosting on GitHub Pages

Step A: Push to GitHub Repository

```
git init
git add .
git commit -m 'feat: complete system design masterclass'
git branch -M main
git remote add origin https://github.com/YOUR_USER/system-design-masterclass.git
git push -u origin main
```

Step B: Enable GitHub Pages

1. Navigate to **GitHub Repo Settings** → **Pages**.
2. Under **Build and deployment**, select **Branch: main** and **Folder: / (root)**.
3. Click **Save**. Your masterclass is live at: https://YOUR_USER.github.io/system-design-masterclass/

3. LinkedIn Algorithm Mastery (How to Maximize Reach)

- **Never Put External Links in the Post Body:** Write '■ *Link in the first comment below!* ■' and comment immediately after publishing.
- **First 60-Minute Rule:** Reply to every comment within 15-30 minutes to trigger the algorithm's recommendation loop.
- **Optimal Schedule:** Post on Tuesday, Wednesday, or Thursday between 7:30 AM – 9:00 AM local time.

4. 8 High-Impact Posts with Dedicated Architecture Diagrams

Post 1: Grand Masterclass Launch

Visual Diagram to Attach: [linkedin_post_assets/Post_01_System_Design_Master_Blueprint.png](#)

■ I spent the last few weeks building an exhaustive, interactive System Design Masterclass covering 616 topics, 108 animated diagrams, and polyglot Python + TypeScript code – and I’m open-sourcing everything today! ■■

■ What’s inside:

- AI/LLM Systems: ChatGPT streaming, ColPali Multimodal RAG, Copilot FIM, Sora video latents.
- 39 Classic & Architect Problems: TinyURL, Uber H3, Ticketmaster locks, Figma CRDTs, Stripe Ledger.
- 11 Legendary Papers: Dynamo, Kafka, Raft, Bigtable, ZooKeeper, and Cassandra.

■■ (Attached: Complete Distributed System Architecture Blueprint)

■ What system design topic do you find most challenging in interviews?

(■ Full interactive curriculum link in first comment! ■)

#SystemDesign #SoftwareEngineering #DistributedSystems #ArtificialIntelligence #OpenSource

Post 2: Amazon Dynamo Quorums & Vector Clocks

Visual Diagram to Attach: [linkedin_post_assets/Post_02_Amazon_Dynamo_Architecture_Diagram.png](#)

■ Why does Amazon never fail when you add an item to your cart – even during data center outages?

In 2007, Amazon published the landmark Dynamo paper, optimizing for 100% write availability (AP under CAP).

4 Core Mechanisms:

- 1■■ Consistent Hashing with 150 Virtual Nodes keeping load variance < 5%.
- 2■■ Sloppy Quorums (N=3, R=2, W=2) guaranteeing read/write overlap.
- 3■■ Hinted Handoff buffering writes during transient node failures.
- 4■■ Vector Clocks tracking causal dependencies with client cart reconciliation.

■■ (Attached: Dynamo Sloppy Quorum Ring & Vector Clock Conflict Resolution)

(■ Python & TypeScript Vector Clock code in first comment! ■)

#DistributedSystems #AmazonDynamo #SystemDesign #HighAvailability

Post 3: Apache Kafka Zero-Copy Throughput

Visual Diagram to Attach: [linkedin_post_assets/Post_03_Kafka_ZeroCopy_vs_Traditional_Sockets.png](#)

■ How does Apache Kafka stream over 1,000,000 messages/second per broker?

Kafka's secret weapon is Linux Kernel Zero-Copy and Sequential Disk I/O.

Traditional Path vs Kafka Zero-Copy:

- Traditional: 4 context switches and 4 data copies (Disk -> OS Cache -> User JVM -> Socket -> NIC).
- Kafka sendfile(): 2 DMA transfers directly from OS Page Cache to NIC Buffer without JVM Heap overhead!

■■ (Attached: Visual Side-by-Side Comparison of Socket Data Transfers)

(■ Full animated diagram & code in first comment! ■)

#ApacheKafka #Performance #Linux #BackendEngineering #SystemDesign

Post 4: Multimodal ColPali RAG Architecture

Visual Diagram to Attach: [linkedin_post_assets/Post_04_Multimodal_ColPali_RAG_Pipeline.png](#)

■ Standard RAG is broken for complex PDFs (tables, charts, slides). Here is how Multimodal ColPali RAG fixes it:

- 1■■ No OCR: Renders 150 DPI page images directly into Vision-Language Models.
- 2■■ 1,024 Patch Embeddings: Captures visual quadrants preserving charts.
- 3■■ CoBERT Late-Interaction MaxSim: Computes token-to-patch max similarity matrix without information loss.

■■ (Attached: ColPali Vision-Language Multi-Vector Ingestion & Retrieval Pipeline)

(■ Full Python MaxSim retrieval algorithm in first comment! ■)
#ArtificialIntelligence #LLM #RAG #MachineLearning #GenerativeAI

Post 5: ChatGPT Token Streaming & Continuous Batching

Visual Diagram to Attach: [linkedin_post_assets/Post_05_ChatGPT_LLM_Streaming_Architecture.png](#)

■ How do you design ChatGPT to serve 100,000,000 daily users with sub-25ms token generation?

Key Architectural Innovations:

- 1■ WebSocket Full-Duplex Gateway with mid-stream abort cancellations.
- 2■ RadixAttention (vLLM) KV-Cache Prefix Sharing: Slashes TTFT by 95%.
- 3■ Continuous Iteration Batching: Re-packs GPU slots token-by-token.
- 4■ 250,000 H100 Global Cluster sizing with 3.35 TB/s HBM3 bandwidth.

■ (Attached: ChatGPT LLM Streaming Architecture & KV-Cache Tree)

(■ Full streaming Python gateway in first comment! ■)

#GenerativeAI #ChatGPT #LLM #SystemDesign #MachineLearning

Post 6: Real-Time Collaborative Figma Canvas

Visual Diagram to Attach: [linkedin_post_assets/Post_06_Figma_Realtime_CRDT_Canvas_Sync.png](#)

■ How does Figma achieve sub-30ms multiplayer canvas sync across designers in different continents?

Unlike locking or central OT, Figma uses Conflict-Free Replicated Data Types (CRDTs):

- LWW-Element-Set: Commutative mathematical operations converge identically on all clients.
- Vector Clocks with Client ID tie-breakers resolve simultaneous edits.
- Ephemeral WebSocket Cursor Fanout with 60 FPS interpolation.

■ (Attached: Figma Real-Time CRDT Convergence Architecture)

(■ Full TypeScript CRDT implementation in first comment! ■)

#Figma #RealTime #CRDT #WebDevelopment #SystemDesign

Post 7: Uber H3 Hexagonal Surge Pricing

Visual Diagram to Attach: [linkedin_post_assets/Post_07_Uber_H3_Surge_Pricing_Architecture.png](#)

■ How does Uber calculate dynamic surge pricing across millions of riders and drivers in real time?

Key Concepts:

- 1■ Uber H3 Hexagonal Grid: Partitions city terrain into uniform hexagons with identical neighbor distances.
- 2■ Real-Time Ingestion: 1M+ drivers emit GPS pings every 4 seconds into Redis Geospatial indexes.
- 3■ Dynamic Multiplier: Calculates supply/demand ratio per hexagon with exponential capping.

■ (Attached: Uber H3 Geospatial Hexagon & Surge Engine)

(■ Full Python surge calculator in first comment! ■)

#Uber #Geospatial #Algorithms #SystemDesign #SoftwareEngineering

Post 8: Raft Distributed Consensus Protocol

Visual Diagram to Attach: [linkedin_post_assets/Post_08_Raft_Consensus_State_Machine.png](#)

■ How do modern distributed databases (etcd, CockroachDB, TiKV) survive node crashes without losing data?

The Secret: Raft Distributed Consensus.

- 150ms-300ms Randomized Election Timers prevent split-vote gridlocks.
- AppendEntries RPC Log Replication across (N/2 + 1) majority quorums.
- Strict Leader Completeness Invariant prevents split-brain corruption.

■ (Attached: Raft State Machine Transition Diagram)

(■ Full Python & TypeScript Raft state machine in first comment! ■)

#DistributedSystems #Raft #Databases #Consensus #SoftwareEngineering